Goroutines and the Main Lifecycle

To create a thread, you simply add the word **"go"** in front of any function call.

```
go add(3 + 1)
```

1. The Main Goroutine Rule

Every Go program has a default Goroutine: the **main()** function. This creates a critical lifecycle rule: **When the main Goroutine completes, all other Goroutines are immediately forced to exit.**

```
func main() {
    go fmt.Println("New Routine")
    fmt.Println("Main Routine")
}
```

If the Go Scheduler schedules the **main** thread to execute first, it reaches the end of the file, and terminates the entire program before the background Goroutine ever gets a chance to run.

2. The time.Sleep() Hack

```
func main() {
   go fmt.Println("New Routine")
   time.Sleep(100 * time.Millisecond) // Force the main thread to pause
   fmt.Println("Main Routine")
}
```

Because the main thread is asleep, the Go Runtime Scheduler says, "Main is blocked, I might as well execute the background Goroutine while I wait!" and prints your background text.

3. Why time.Sleep() is Terrible Practice

While sleeping works for quick testing, **you must never use this in real production code.**

- We are blindly guessing that 100 milliseconds is enough time for the background task to finish.
- We have no idea what the **OS** is doing. The OS might pause your entire Go program for 100ms to allocate CPU resources to a background antivirus scan. By the time Go wakes up, the delay is over, and your Goroutine still hasn't executed!

Expert Takeaway

Because we cannot predict the exact speed of the OS or the Go Runtime Scheduler, we must use tools that explicitly tell the main thread exactly when a background task finishes, regardless of whether it takes 1 millisecond or 10 seconds.